DeVinci Blockchain

Git & GitHub Cheat sheet

DVB WORKSHOP
22.10.2026 · IIM NANTERRE

KEEP IT OPEN WHILE YOU CODE

BLUE = REPLACE WITH YOUR OWN VALUE

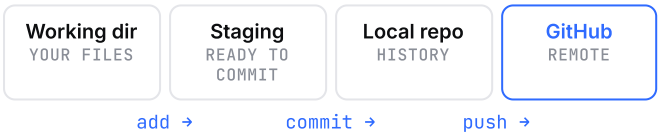
01 Set up (once per computer)

`git config --global user.name "First Last"`	Your name, shown on every commit.
`git config --global user.email "you@mail.com"`	The **same e-mail** as your GitHub account.
`git config --global init.defaultBranch main`	New repos start on the **main** branch.
`git config --global core.editor "code --wait"`	Opens VS Code instead of Vim for messages.
`git config --list`	Check your configuration.

02 Start a project

`git init`	Turns the current folder into a Git repo.
`git clone <url>`	Copies a remote repo to your machine.
`git --version`	Checks that Git is installed.

03 The basic cycle



`git status`	Where am I? **Type it all the time.**
`git add file`	Adds a file to the staging area.
`git add .`	Adds every change in the folder.
`git commit -m "message"`	Saves a "snapshot" of the staging area.
`git diff`	Changes not added yet.
`git diff --staged`	Changes ready to be committed.
`git log --oneline --graph --all`	The history as a compact graph.

04 Branches

`git branch`	Lists branches (* = current branch).
`git switch -c feature/my-card`	Creates a branch and moves onto it.
`git switch main`	Goes back to main.
`git merge feature/my-card`	Merges that branch **into the current one**.
`git branch -d feature/my-card`	Deletes a branch that is already merged.

05 Working with GitHub (remote)

`git remote -v`	Shows the remote repos (origin...).
`git remote add origin <url>`	Links your local repo to GitHub.
`git push -u origin branch`	First push of a branch (then just `git push`).
`git pull`	Downloads **and merges** new changes.
`git fetch`	Downloads without merging anything.

06 Undo

`git restore file`	Discards changes **not added yet** (gone for good!).
`git restore --staged file`	Unstages a file, keeps the changes.
`git commit --amend -m "msg"`	Fixes the last commit (before pushing).
`git revert <hash>`	New commit that undoes an old one. **Safe** even after a push.
`git reset --soft HEAD~1`	Undoes the last commit, keeps the changes staged.

⚠ **Careful:** `amend` and `reset` rewrite history. Never use them on a commit that is already pushed and shared: use `revert`.

The Builders Wall
Add your card with a Pull Request:
`github.com/alibenyezza/dvb-git-workshop`

07 Fix a merge conflict

Git can't merge two changes to the same line, so it shows you both versions:

```
<<<<<<< HEAD
Welcome to the Git workshop!   ← your version
=======
Welcome to the DVB workshop!   ← the other branch
>>>>>>> feature/title
```

1. **Open the file** (VS Code highlights the conflict).
2. **Keep the right version** (or combine them) and **delete the 3 markers**.
3. `git add file` to mark the conflict as resolved.
4. `git commit` to finish the merge.

`git merge --abort`	Panicking? Cancels the merge and goes back to how things were.

08 Contribute with a Pull Request

`Fork` → `Clone` → `Branch` → `Commit` → `Push` → `PR` → `Review` → `Merge`

`"Fork" button on GitHub`	Creates your own copy of the repo on your account.
`git clone https://github.com/you/dvb-git-workshop.git`	Clone **your fork**, not the original.
`git switch -c add/username`	One branch for your contribution.
`builders/username.json`	Create your card (copy `_template.json`).
`git add builders/username.json` `git commit -m "feat: add username to the wall"`	Saves your card.
`git push -u origin add/username`	Sends the branch to your fork.
`"Compare & pull request"`	On GitHub. Wait for the **validate** ✓ check, then the merge.

09 Write good commits

Conventional Commits format: `type: short description`

`feat:`	new feature — `feat: add login page`
`fix:`	bug fix — `fix: broken login link`
`docs:`	documentation — `docs: update README`
`chore:`	maintenance — `chore: bump dependencies`
`refactor:`	rewrite without changing behaviour

Short (< 72 characters), imperative mood, **one commit = one change**.

10 Versions & tags

SemVer: `MAJOR.MINOR.PATCH` — breaking change · new feature · bug fix.

`git tag -a v1.0.0 -m "First release"`	Creates an annotated tag on the current commit.
`git tag`	Lists tags.
`git push origin v1.0.0`	Pushes a tag (a plain `git push` doesn't send tags).
`git push --tags`	Pushes every tag.

11 Help!

`Stuck in Vim`	**Esc** then `:wq` + Enter (save) or `:q!` (quit without saving).
`Author identity unknown`	Set `user.name` and `user.email` (section 01).
`! [rejected] … (fetch first)`	The remote moved on: `git pull --rebase` then `git push`.
`You are in 'detached HEAD'`	Go back to a branch: `git switch -` or `git switch main`.
`not a git repository`	Wrong folder: `cd` into your repo (or `git init`).
`Authentication failed`	GitHub no longer accepts passwords: sign in through the browser on your 1st push, or run `gh auth login`.
`Permission denied (403)`	You're pushing to the original repo: push to **your fork**.

Keep learning

Official docs: `git-scm.com/book` · Practice: `learngitbranching.js.org`
Student Developer Pack: `education.github.com/pack`